

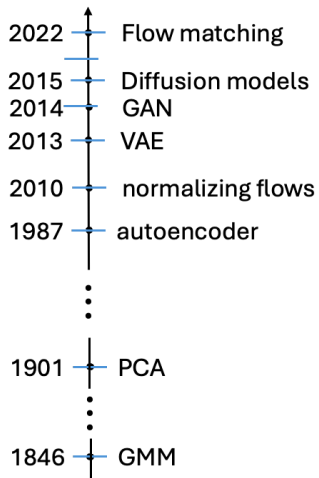
COMM114: Generative AI - Diffusion Models (part 2)

Guoqiang Zhang

September 2025

Diffusion models

Diffusion Models was invented in 2015.



Objective of this lecture

- In this lecture, we extend the ODE to stochastic differential equation (SDE) that involves Brownian motion.
- We then explain how to generate new data samples from the SDE.

Remark:

- No need to retrain a new model for the SDE.
- The model trained for the ODE can be reused for sampling from SDE.

Background on Brownian motion (cont.)

- A standard scalar Brownian motion (alternatively referred to as Wiener process) on the interval $[0, T]$ is a random variable W_t that depends continuously on $t \in [0, T]$ and satisfies the following:
 - $W_0 = 0$
 - For $0 \leq s < t \leq T$,

$$W_t - W_s \sim \sqrt{t - s} \mathcal{N}(0, 1)$$

- For $0 \leq s < t < u < v \leq T$, $W_t - W_s$ and $W_v - W_u$ are independent.

Background on Brownian motion

- We use dW_t to denote the increment of the Brownian motion over a very small time interval $dt = [t, t + dt]$. Then we have

$$dW_t = \sqrt{dt}\mathcal{N}(0, 1).$$

- As a result, W_t at time t can be approximated as

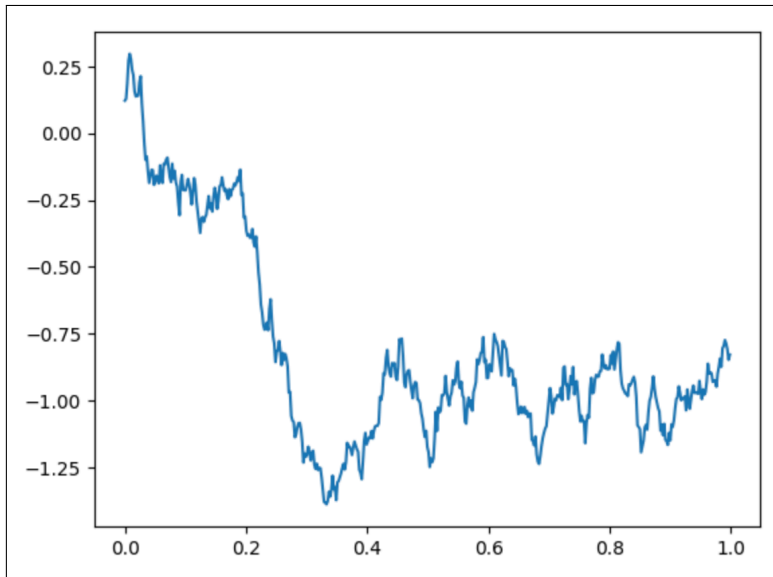
$$\begin{aligned} W_t &= \int_{t=0}^{t=1} dW_t \\ &\approx \sum_{i=0}^{N-1} \int_{t_i}^{t_{i+1}} dW_t \\ &= \sum_{i=0}^{N-1} \sqrt{t_{i+1} - t_i} \mathcal{N}(0, 1) \end{aligned}$$

Background on Brownian motion (cont.)

Simulation of Brownian motion

```
1 import torch
2 import matplotlib.pyplot as plt
3
4 T = 1      #Simulation over time interval [0,1]
5 N = 500
6 dt = T/N   # Define the small time step
7
8 dW = torch.sqrt(torch.tensor(dt))*torch.randn(1,N)
9
10 W = torch.cumsum(dW,dim=1)
11
12 timesteps = torch.linspace(0, T, steps=N).view(1,N)
13
14 plt.plot(timesteps.numpy()[0, 0:-1], W.numpy()[0, 0:-1])
15 plt.show()
```

Background on Brownian motion (cont.)



Remark: Different rand seeds will lead to different noise samples over time.

Stochastic differential Equation (SDE)

- Scalar SDE over time interval $[0, T]$

$$dx_t = f_t(x_t)dt + \sigma_t dW_t \quad [dW \text{ refers to the scalar Brownian motion}]$$

- Vector SDE over time interval $[0, T]$

$$d\mathbf{x}_t = f_t(\mathbf{x}_t)dt + \sigma_t d\mathbf{W}_t \quad [d\mathbf{W} \text{ refers to the vector Brownian motion}]$$

Remark: At time step t , the state $\mathbf{x}_t$ is driven by both a deterministic function $f_t(\cdot)$ and the Brown motion.

Simulating an SDE

Algorithm 1 SDE simulation

- 1: Input: $\mathbf{x}_0$, number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{T}{k}$
 - 4: **for** $i = 1, \dots, k$ **do**
 - 5: Draw a sample $\epsilon \sim \mathcal{N}(0, \mathbf{I}_d)$
 - 6: $\mathbf{x}_{t+h} = \mathbf{x}_t + hf_t(\mathbf{x}_t) + \sigma_t \sqrt{h} \epsilon$
 - 7: Set $t \leftarrow t + h$
 - 8: **end for**
 - 9: return $\mathbf{x}_T$
-

[Add additional Gaussian noise with
var = h scaled by coefficient σ_t]

SDE (cont.)

Applications SDE

- **Generative AI:** data sample generation in Diffusion models (topic of this lecture).
- **finance:** modeling of the stock value dynamics.
- **Physics:** SDEs are employed to describe physical systems that are subject to thermal fluctuations.
- **Mathematical biology:** SDEs can model the behavior of biological populations and other complex systems

Recap from last lecture

- Let PDF of $\mathbf{X}_0$ be $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (i.e., $p_{\mathbf{X}_0}(\mathbf{x}_0) = p_{init}(\mathbf{x})$)
- Let PDF of $\mathbf{X}_1$ be $p_{data}(\mathbf{x})$ (i.e., $p_{\mathbf{X}_1}(\mathbf{x}_1) = p_{data}(\mathbf{x})$)
- Define $\mathbf{X}_t = \beta_t \mathbf{X}_0 + \alpha_t \mathbf{X}_1$ where α_t and β_t satisfy:
 - α_t and β_t are scalar differentiable functions over $t \in [0, 1]$.
 - When $t = 0$, $\alpha_{t=0} = 0$ and $\beta_{t=0} = 1$.
 - When $t = 1$, $\alpha_{t=1} = 1$ and $\beta_{t=1} = 0$.
- Consider the following ODE

$$\frac{d\mathbf{X}_t}{dt} = \left(\beta_t^2 \frac{d \log \alpha_t}{dt} - \frac{1}{2} \frac{d \beta_t^2}{dt} \right) \nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{X}_t | \mathbf{X}_1) + \frac{d \log \alpha_t}{dt} \mathbf{X}_t \quad (1)$$

- It can be shown that $\mathbf{X}_t$, the solution to the above ODE, transforms the distribution $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ at $t = 0$ to the data distribution p_{data} at $t = 1$.
- In other words, the probability path of $\mathbf{X}_t$ satisfy the boundary probability conditions.

Recap from last lecture (cont.)

Prior knowledge:

It is known that the conditional distribution $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$ is Gaussian. As a result, the score function $\nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$ in (1) takes an alternative form:

$$\nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = -\frac{\mathbf{x}_t - \alpha_t \mathbf{x}_1}{\beta_t^2} = -\frac{\mathbf{x}_0}{\beta_t}$$

where $\mathbf{x}_0$ is the Gaussian noise when constructing $\mathbf{x}_t$.

- **Input:** data samples $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.
- **Assumption:** Assume $\{\mathbf{x}_i\}_{i=1}^n$ are drawn from the data distribution $p_{data}(\cdot)$.
- A neural network $\hat{\epsilon}_t^\theta(\cdot)$ is trained by optimizing

$$\begin{aligned} & \arg \min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0,1], \mathbf{x} \sim p_{data}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)} \left[\left\| \hat{\epsilon}_t^\theta(\mathbf{x}_t) - \epsilon \right\|^2 \right] \quad [\epsilon \text{ corresponds to } \mathbf{x}_0] \\ & = \arg \min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0,1], \mathbf{x} \sim p_{data}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)} \left[\left\| \hat{\epsilon}_t^\theta(\alpha_t \mathbf{x} + \beta_t \epsilon) - \epsilon \right\|^2 \right] \end{aligned} \quad (2)$$

Recap from last lecture (cont.)

- Once the neural network $\hat{\epsilon}_t^\theta(\cdot)$ is trained, the ODE (1) can be approximated in terms of $\hat{\epsilon}_t^\theta(\cdot)$ as:

$$\begin{aligned} \frac{d\mathbf{x}_t}{dt} &= \left(\beta_t^2 \frac{d \log \alpha_t}{dt} - \frac{1}{2} \frac{d \beta_t^2}{dt} \right) \nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{x}_t | \mathbf{x}_1) + \frac{d \log \alpha_t}{dt} \mathbf{x}_t \\ \Leftrightarrow \frac{d\mathbf{x}_t}{dt} &= \left(\beta_t^2 \frac{d \log \alpha_t}{dt} - \frac{1}{2} \frac{d \beta_t^2}{dt} \right) \frac{-\hat{\epsilon}_t^\theta(\mathbf{x}_t)}{\beta_t} + \frac{d \log \alpha_t}{dt} \mathbf{x}_t \\ \Leftrightarrow d\mathbf{x}_t &= (d\beta_t - \beta_t d \log \alpha_t) \hat{\epsilon}_t^\theta(\mathbf{x}_t) + d \log \alpha_t \mathbf{x}_t \text{ where } \mathbf{x}_0 \sim p_{init} \end{aligned} \quad (3)$$

Remark: With the neural network $\hat{\epsilon}_t^\theta(\cdot)$, new data samples can be generated by simulating the ODE (3) via Euler method.

SDE Formulation

- Let PDF of $\mathbf{X}_0$ be $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (i.e., $p_{\mathbf{X}_0}(\mathbf{x}_0) = p_{init}(\mathbf{x})$)
- Let PDF of $\mathbf{X}_1$ be $p_{data}(\mathbf{x})$ (i.e., $p_{\mathbf{X}_1}(\mathbf{x}_1) = p_{data}(\mathbf{x})$)
- Define $\mathbf{X}_t = \beta_t \mathbf{X}_0 + \alpha_t \mathbf{X}_1$ where α_t and β_t satisfy:
 - α_t and β_t are scalar differentiable functions over $t \in [0, 1]$.
 - When $t = 0$, $\alpha_{t=0} = 0$ and $\beta_{t=0} = 1$.
 - When $t = 1$, $\alpha_{t=1} = 1$ and $\beta_{t=1} = 0$.
- One can construct an SDE in the form of

$$d\mathbf{X}_t = \left(\beta_t^2 d \log \alpha_t - \frac{1}{2} d\beta_t^2 + \frac{\sigma_t^2 dt}{2} \right) \nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{X}_t | \mathbf{X}_1) + d \log \alpha_t \mathbf{X}_t + \sigma_t d\mathbf{W}_t \quad (4)$$

- It can be shown that $\mathbf{X}_t$, the solution to the above SDE, transforms the distribution $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ at $t = 0$ to the data distribution p_{data} at $t = 1$.
- In other words, the probability path of $\mathbf{X}_t$ satisfy the boundary probability conditions.

SDE Formulation (cont.)

The SDE form in a diffusion model:

$$d\mathbf{X}_t = \left(\beta_t^2 d \log \alpha_t - \frac{1}{2} d \beta_t^2 + \frac{\sigma_t^2 dt}{2} \right) \nabla_{\mathbf{X}_t} \log p_{t|1}(\mathbf{X}_t | \mathbf{X}_1) + d \log \alpha_t \mathbf{X}_t + \sigma_t d\mathbf{W}_t$$

- σ_t can be any continuous function of t .
- When σ_t is set to 0, it reduces to the ODE form given in Equation (1).

SDE Formulation (cont.)

- Similar to that of ODE in a diffusion model, once the neural network $\hat{\epsilon}_t^\theta(\cdot)$ is trained, the SDE can be approximated in terms of the DNN model as

$$d\mathbf{X}_t = \left(\beta_t^2 d \log \alpha_t - \frac{1}{2} d\beta_t^2 + \frac{\sigma_t^2 dt}{2} \right) \nabla_{\mathbf{X}_t} \log p_{t|1}(\mathbf{X}_t | \mathbf{X}_1) + d \log \alpha_t \mathbf{X}_t + \sigma_t d\mathbf{W}_t$$

$$\Leftrightarrow d\mathbf{x}_t = \left(\beta_t^2 d \log \alpha_t - \frac{1}{2} d\beta_t^2 + \frac{\sigma_t^2 dt}{2} \right) \frac{-\hat{\epsilon}_t^\theta(\mathbf{x}_t)}{\beta_t} + d \log \alpha_t \mathbf{x}_t + \sigma_t d\mathbf{W}_t$$

$$\Leftrightarrow d\mathbf{x}_t = \left(d\beta_t - \beta_t d \log \alpha_t - \frac{\sigma_t^2 dt}{2\beta_t} \right) \hat{\epsilon}_t^\theta(\mathbf{x}_t) + d \log \alpha_t \mathbf{x}_t + \sigma_t d\mathbf{W}_t \text{ where } \mathbf{x}_0 \sim p_{init}$$

(5)

Sampling via SDE

- At time step t , first generate a Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then $\mathbf{x}_{t+h}$ can be computed in terms $\mathbf{x}_t$ and ϵ as

$$\mathbf{x}_{t+h} - \mathbf{x}_t = \left(\beta_{t+h} - \beta_t - \beta_t \frac{\alpha_{t+h} - \alpha_t}{\alpha_t} - \frac{\sigma_t^2 h}{2\beta_t} \right) \hat{\epsilon}_t^\theta(\mathbf{x}_t) + \frac{\alpha_{t+h} - \alpha_t}{\alpha_t} \mathbf{x}_t + \sigma_t \sqrt{h} \epsilon$$

$$\Leftrightarrow \mathbf{x}_{t+h} = \left(\beta_{t+h} - \beta_t \frac{\alpha_{t+h}}{\alpha_t} - \frac{\sigma_t^2 h}{2\beta_t} \right) \hat{\epsilon}_t^\theta(\mathbf{x}_t) + \frac{\alpha_{t+h}}{\alpha_t} \mathbf{x}_t + \sigma_t \sqrt{h} \epsilon$$

$$\Leftrightarrow \mathbf{x}_{t+h} = \left(\beta_{t+h} - \frac{\sigma_t^2 h}{2\beta_t} \right) \hat{\epsilon}_t^\theta(\mathbf{x}_t) + \alpha_{t+h} \frac{(\mathbf{x}_t - \beta_t \hat{\epsilon}_t^\theta)}{\alpha_t} + \sigma_t \sqrt{h} \epsilon$$

Remark: In practice, σ_t can be chosen as a function of β_t . For example, $\sigma_t^2 = 2\beta_t h$.

Sampling via SDE

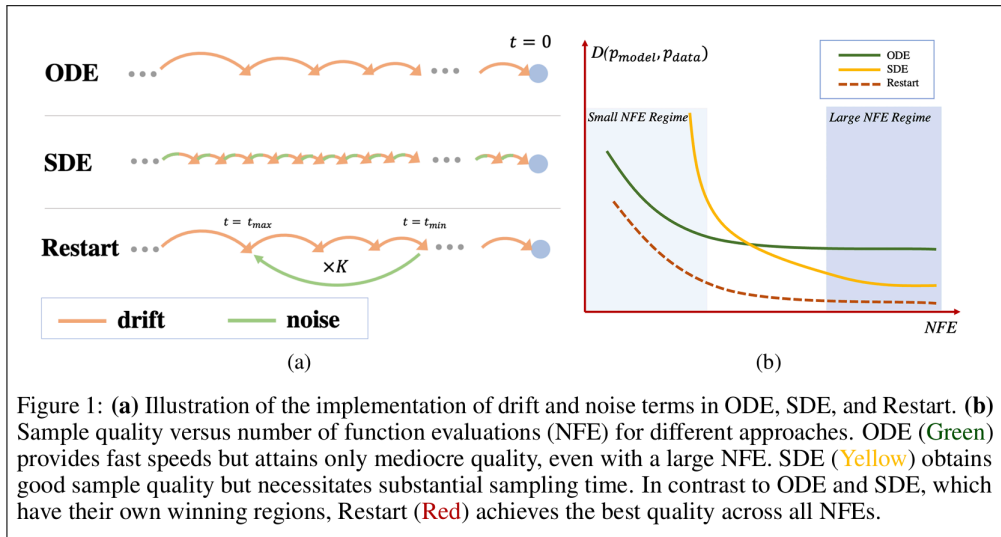
Algorithm 2 Sampling via SDE

- 1: Input: neural network $\hat{\epsilon}_t^\theta$, number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{1}{k}$
 - 4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 5: **for** $i = 1, \dots, k$ **do**
 - 6: Draw a Gaussian noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
 - 7: $\mathbf{x}_{t+h} = \left(\beta_{t+h} - \frac{\sigma_t^2 h}{2\beta_t} \right) \hat{\epsilon}_t^\theta(\mathbf{x}_t) + \alpha_{t+h} \frac{(\mathbf{x}_t - \beta_t \hat{\epsilon}_t^\theta)}{\alpha_t} + \sigma_t \sqrt{h} \epsilon$
 - 8: Set $t \leftarrow t + h$
 - 9: **end for**
 - 10: return $\mathbf{x}_{t=1}$
-

SDE sampler versus ODE sampler

SDE (Stochastic Differential Equation) sampling is better when sample quality is the priority and sampling time is less critical, as the addition of randomness helps it achieve higher quality in tasks like image generation. Conversely, ODE (Ordinary Differential Equation) sampling is better when speed is the priority,

Restart Sampler



Remark: [paper-link on Restart sampler](#)

References

- Lecture notes by MIT
- Tutorial on Flow Matching for Generative Modeling
- Introduction to Brownian Motion

Questions?